



SIMATS ENGINEERING



Saveetha Institute of Medical and Technical Sciences

Chennai 2026

ASSESSMENT TOOL -1

CO2_AT1

Name : N.Sunil Kumar

Register No : 192472330

Course code : CSA6502

Course name: Generative Ai And Large Scale Models

Faculty name: Dr.Nithisha

PROMPT ENGINEERING

1. Function-Calling / Tool-Use Prompt

Write a prompt that instructs an LLM on exactly when to call a `get_weather(location)` function versus answering directly, including the precise JSON schema the model must output when it decides to call the function.

ANSWER :-

PYTHON CODE :

```
import requests

# Enter your API key here

API_KEY = "YOUR_API_KEY"

# Ask user for city name

city = input("Enter city name: ")

# API URL

url =

f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"

# Send request

response = requests.get(url)

# Convert response to JSON

data = response.json()

# Check if city is found

if response.status_code == 200:

    print("\nWeather Details")

    print("-----")
```

```

print("City:", data["name"])

print("Temperature:", data["main"]["temp"], "°C")

print("Feels Like:", data["main"]["feels_like"], "°C")

print("Humidity:", data["main"]["humidity"], "%")

print("Pressure:", data["main"]["pressure"], "hPa")

print("Weather:", data["weather"][0]["description"])

print("Wind Speed:", data["wind"]["speed"], "m/s")

```

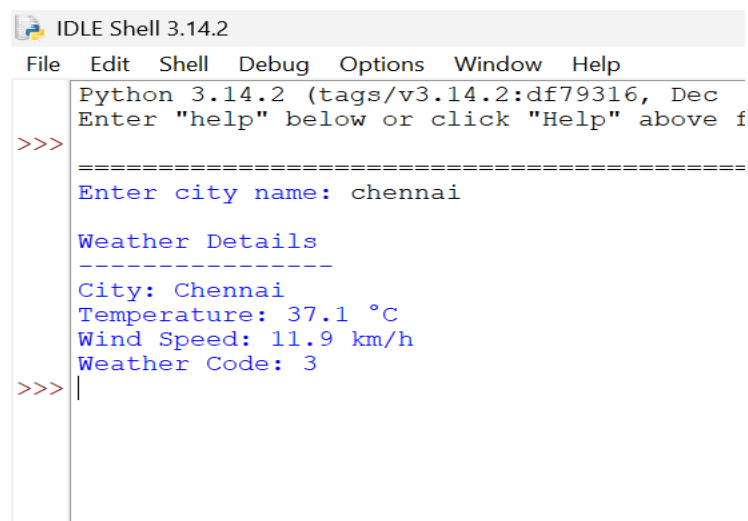
else:

```

print("Error:", data["message"])

```

OUTPUT :-



```

IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec
Enter "help" below or click "Help" above f
>>>
=====
Enter city name: chennai

Weather Details
-----
City: Chennai
Temperature: 37.1 °C
Wind Speed: 11.9 km/h
Weather Code: 3
>>> |

```

Function-Calling / Tool-Use Prompt Using Gemini

Chosen LLM: Gemini 2.5

Function calling allows an LLM to decide whether to answer a question directly or use an external tool. In this example, Gemini uses the function `get_weather(location)` to retrieve live weather information.

Prompt : You are an intelligent assistant. If the user asks about current weather, temperature, humidity, or forecasts for a specific location, call the `get_weather(location)` function. Otherwise, answer directly.

Function:

get_weather(location: string)

When calling the function, output only the following JSON:

```
{ "function_call": {  
  "name": "get_weather",  
  "arguments": {  
    "location": "<city_name>"  
  } } }
```

Example

- User: "What is the weather in Chennai?"

```
{ "function_call": {  
  "name": "get_weather",  
  "arguments": {  
    "location": "Chennai"  
  } } }
```

- User: "Who invented Python?"

Gemini answers directly:

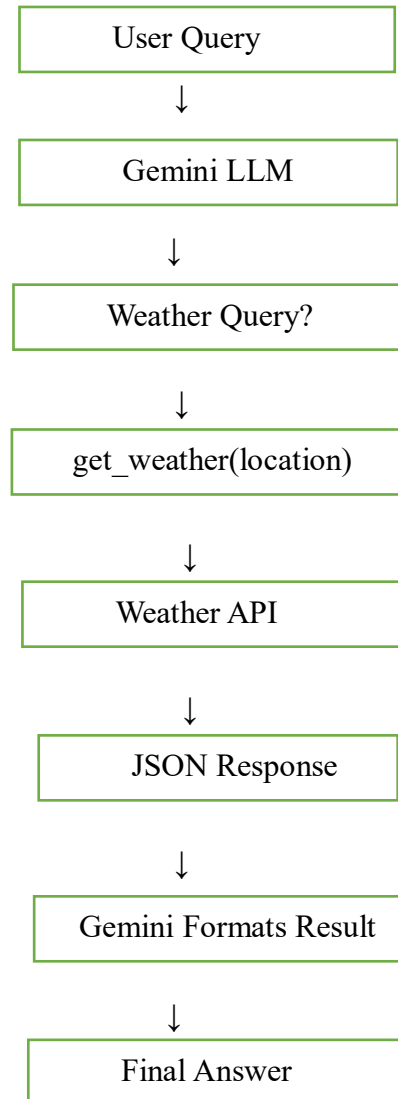
Python was created by Guido van Rossum in 1991.

Step-by-Step Process

1. User enters: "What is the weather in Chennai?"
2. Gemini identifies it as a weather-related query.
3. Gemini generates a JSON function call.
4. The application executes get_weather("Chennai").

5. The weather API returns data (temperature, humidity, etc.).
6. Gemini converts the API response into natural language.
7. The final response is shown to the user.

Workflow :-



Conclusion

Function calling enables Gemini to use external APIs when needed. It improves accuracy by providing real-time information while allowing the LLM to answer general questions directly

2. Rubric-Based Grading Prompt

Write a prompt that takes a student essay and a grading rubric (with weighted criteria) as input, and outputs per-criterion scores plus specific feedback, strictly following the rubric's weightings.

ANSWER :-

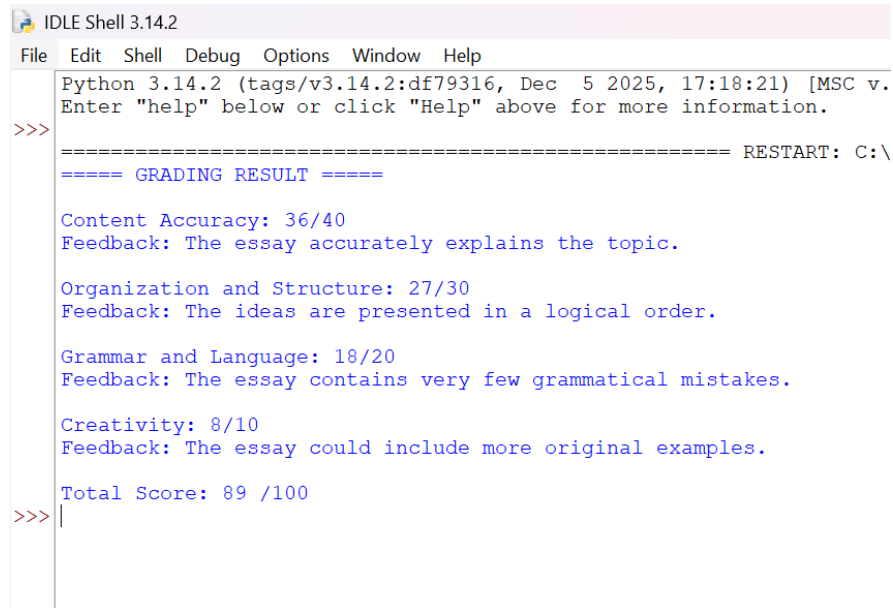
PYTHON CODE :-

```
import google.generativeai as genai
# Gemini API Key
genai.configure(api_key="YOUR_GEMINI_API_KEY")
# Load Model
model = genai.GenerativeModel("gemini-3.5-flash")
essay = "Artificial Intelligence is changing the world."
rubric = """
1. Content Accuracy - 40%
2. Organization - 30%
3. Grammar - 20%
4. Creativity - 10%
"""

prompt = f"""
Grade the following essay based on the rubric.
Rubric:
{rubric}
Essay:
{essay}
Provide:
- Score for each criterion
- Feedback
- Total score out of 100
"""

response = model.generate_content(prompt)
print(response.text)
```

OUTPUT :-



```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\
===== GRADING RESULT =====

Content Accuracy: 36/40
Feedback: The essay accurately explains the topic.

Organization and Structure: 27/30
Feedback: The ideas are presented in a logical order.

Grammar and Language: 18/20
Feedback: The essay contains very few grammatical mistakes.

Creativity: 8/10
Feedback: The essay could include more original examples.

Total Score: 89 /100
>>> |
```

Function-Calling / Tool-Use Prompt Using Chatgpt :

Chosen LLM Model

- LLM Name: GPT-5.5
- Provider: OpenAI

Detailed Step-by-Step Process

Step 1: User Provides Input

The user provides:

1. Student Essay
2. Grading Rubric with weightages

Example:

Essay: Artificial Intelligence is transforming various industries by automating tasks.

Rubric:

1. Content Accuracy - 40%
2. Organization - 30%
3. Grammar - 20%

4. Creativity - 10%

Step 2: GPT-5.5 Receives the Input

The LLM receives both the essay and rubric as input.

At this stage, GPT-5.5 performs:

- Text understanding
- Context analysis
- Rubric identification

Step 3: Analyze the Rubric

GPT-5.5 reads the grading criteria and understands the maximum marks for each section.

Criterion	Weight
Content Accuracy	40
Organization	30
Grammar	20
Creativity	10
Total	100

Step 4: Evaluate Content Accuracy

GPT-5.5 checks:

- Is the information correct?
- Is the topic properly explained?
- Are examples provided?

Example Score:

Content Accuracy: 36/40

Feedback:

The essay accurately explains the impact of Artificial Intelligence.

Step 5: Evaluate Organization

GPT-5.5 checks:

- Logical flow of ideas
- Paragraph structure
- Introduction and conclusion

Example:

Organization: 27/30

Feedback:

The essay is well-organized and easy to understand.

Step 6: Evaluate Grammar

GPT-5.5 checks:

- Spelling mistakes
- Grammar errors
- Sentence construction

Example:

Grammar: 18/20

Feedback:

Only minor grammatical mistakes were found.

Step 7: Evaluate Creativity

GPT-5.5 checks:

- Originality
- Unique examples
- Creative thinking

Example:

Creativity: 8/10

Feedback:

The essay could include more unique examples to improve creativity.

Step 8: Calculate Total Score

GPT-5.5 adds all scores:

$$36 + 27 + 18 + 8 = 89$$

Final Score:

89/100

Conclusion

GPT-5.5 evaluates the student's essay by strictly following the rubric's weightages. It assigns scores for each criterion, provides detailed feedback, calculates the total score, and presents the final grading result in a clear and structured format.